

mpc_cuda: GMP/MPFR/MPC の CUDA 移植

— 作成手順・実装上の困難・性能評価 —

tkouya

2026 年 6 月 11 日

概要

本報告は、任意精度演算ライブラリ群 GNU MP (GMP) / MPFR / MPC を NVIDIA GPU の CUDA カーネル内部で動作させたライブラリ **mpc_cuda** の作成手順、移植過程で直面した実装上の困難とその解決、および基本線形計算と初等関数の性能評価をまとめたものである。さらに、コンパイル時に精度を固定するレジスタ常駐型の高速パス `cu_frexp<PB>/cu_fcomplex<PB>` を新設し、精度 128–8192 ビットでの GPU・CPU 性能のクロスオーバーを定量的に示す。主対象ハードウェアは NVIDIA GB10 (compute capability 12.1, `sm_121`, CUDA 13) であり、さらにランタイム精度デモを NVIDIA H100 NVL (compute capability 9.0, `sm_90`) とクロス比較する。全結果はホストの MPFR/MPC とビット一致（または初等関数で ≤ 1 ULP）である。

目次

1	はじめに	1
2	全体設計と作成手順	2
2.1	出発点となる上流ソース	2
2.2	変換スクリプト方式	2
2.3	層別の段階	2
3	実装上の困難とその解決	2
3.1	コンパイルを通すための適応	2
3.2	デバイスで正しく動かすための適応	3
3.3	最大の難関：デバイスでの <code>mpfr_div</code> 誤コンパイル	4
3.4	スタック・アリーナ・並列度	4
3.5	共存： <code>cu_</code> 名前空間と <code>umbrella</code> ヘッダ	5
3.6	その他のバグと教訓	5
4	コンパイル時固定精度の高速パス	5

4.1	設計	6
4.2	初等関数	6
5	性能評価	6
5.1	AXPY (実数・複素)	6
5.2	基本線形計算 (行列ベクトル積・行列積)	7
5.3	初等関数	7
5.4	固定精度高速パスと精度スweep	8
5.5	CPU 上での固定精度パス	10
5.6	考察	11
5.7	クロスプラットフォーム比較: GB10 vs H100 (総括)	11
6	まとめと今後	14

1 はじめに

GPU 上で任意精度演算を行いたいという要求は、悪条件問題の数値線形代数や高精度シミュレーションなどで根強い。既存の GPU 向け多倍長ライブラリ (例: CUMP は GMP の mpf を移植) は存在するが、丸めの規約が緩く、複素数や初等関数を欠くものが多い。本プロジェクトの目標は、**正しく丸められる MPFR の実数意味論とその上の複素数層 MPC** を、可能な限り**忠実に** (再実装ではなく上流ソースの移植として) CUDA カーネル内で動かすことである。具体的な到達目標は、 $y = ax + y$ (AXPY) を実数 (MPFR)・複素 (MPC) の両方で GPU 上で実行し、CPU 版とビット一致させ、かつ十分に高速化することであった。

本報告の構成は次の通りである。第 2 節で全体設計と作成手順 (変換スクリプト方式と層別の段階) を述べ、第 3 節で移植上の困難と解決を詳述する。第 4 節でコンパイル時固定精度の高速パスを、第 5 節で性能評価 (AXPY, 基本線形計算, 初等関数, 精度スweep) を示し、第 6 節でまとめる。

2 全体設計と作成手順

2.1 出発点となる上流ソース

移植の土台として、GMP のサブセット実装である **mini-gmp** (gmp-6.3.0/mini-gmp) を採用した。mini-gmp は単一の C ソースで mpz (多倍長整数) と mpn (低レベルリム列) の主要部を提供する。MPFR-4.2.2 は `--with-mini-gmp` オプションで mini-gmp 上にビルドでき、不足する mpn ルーチン (mpn_divrem 等) は MPFR 同梱の `mpfr-mini-gmp.c` が補う。すなわち**フル GMP の内部実装を必要としない**。MPC-1.4.1 は MPFR の上の薄い複素層であり、同じ土台に載る。

2.2 変換スクリプト方式

上流を手作業でフォークすると上流更新の取り込みが困難になる。そこで本プロジェクトでは**再実行可能な変換スクリプト** (tools/cudafy_*.py) でデバイス適応を行う方針を採った。

- tools/cudafy_minigmp.py: pristine な mini-gmp.{c,h} を読み, デバイス到達関数に __host__ __device__ を付与した include/mpc_cuda/cuda_minigmp.h と src/cuda_minigmp.cu を生成する。ホスト専用関数 (stdio/realloc/ctype を使うもの) は字句走査で自動判定する。
- tools/cudafy_mpfr.py: MPFR の 263 個のソースに同様の適応を施す。
- tools/cudafy_mpc.py: MPC の 90 個のソースに適応を施す (MPFR 用の機構を再利用)。

これにより上流の構造を保ったまま, スクリプトの再実行で更新を再適用できる。

2.3 層別の段階

開発は次の 4 段階 (+ 2b) で進めた。各段階はホスト版とビット一致を確認しながら積み上げた。

表 1 層別の開発段階

段階	層	内容
1	mini-gmp	mpz/mpn をデバイスで動作。add/sub/mul/t-div_qr/pow_ui/get_str をホストとビット一致 (256 スレッド)。
2	MPFR (忠実移植)	実 mpfr-4.2.2 の init2/set/mul/add/get_d がカーネル内で動作。
2b	MPFR 超越関数	pi/exp/log/sin/cos/atan/sqrt が動作 (後述の mpfr_div 代替が必要)。
3	MPC (複素)	実 mpc-1.4.1 の init2/set_d.d/mul/add が動作。
4	AXPY デモ・ベンチ	実・複素の AXPY を GPU/CPU で時間・精度比較。

3 実装上の困難とその解決

「コンパイルが通る」ことと「デバイスで正しく動く」ことは別問題である。本節では移植の核心となった困難を分類して述べる。

3.1 コンパイルを通すための適応

1. **C11 _Noreturn**: nvcc の C++ フロントエンドが `_Noreturn` を拒否する。ホスト版にある `-DMPFR_HAVE_NORETURN` を外す。
2. **インラインアセンブリ**: `mpfr-longlong.h` の x86_64 用 `umul_ppmm` 等はデバイスで無効。`-DNO_ASM` で移植版 C フォールバックを選択する。
3. **realloc の不在**: CUDA デバイスコードに `realloc` が無い。`malloc+memcpy+free` で擬似的に実装する (`mg_cuda_realloc`)。
4. **C99 _Complex**: デバイスコードに `_Complex` が無い。MPC の `mpc.h` の `#if 1` (`_MPC_HAVE_COMPLEX_H` 定義) を `#if 0` に変え、`mpc_set_dc/get_dc` を無効化する。
5. **gmp.h シム**: MPC の `mpc.h` は `gmp.h` を `#include` する。mini-gmp を指す `gmp.h` シムを生成し、mini-gmp に無い `gmp_randstate_t/mpf_t/mpq_t` の最小スタブを宣言してプロトタイプを通す。
6. **トークン貼り付け**: MPC の `set_x.c/set_x_x.c` は `mpfr_set_##type` で全 setter を 1 つの翻訳単位に展開し、mini-gmp に無い `mpfr_set_f/_q` を巻き込む。これらにトラップするデバイススタブを注入してコンパイルを通す (未使用なら呼ばれない)。

3.2 デバイスで正しく動かすための適応

ここからが本質的に難しい部分である。コンパイルが通っても、ホスト前提のグローバル状態・関数ポインタ・最適化がデバイスで誤動作する。

1. **ルックアップテーブル**: `invert_limb_table` や `__clz_tab` 等のファイルスコープ `static const` 配列はデバイスから参照できない。MPFR_RODATA タグ (デバイスパスで `__device__`, ホストパスで空) を付け、1 つの定義で両パスを賄う。
2. **グローバル状態**: `__gmpfr_emin/emax/flags` や既定精度・丸めは `__device__ __managed__` (ホスト・デバイス共有) にする。
3. **assert/abort**: `mpfr_assert_fail` (`fprintf+abort`) や `gmp_die` はデバイスで `__trap()` に置換する。
4. **__builtin_unreachable の誤コンパイル**: `MPFR_HAVE_BUILTIN_UNREACHABLE` を有効にすると、その `__builtin_constant_p/_builtin_unreachable` 経路が `MPFR_ASSERTN` をデバイスで誤コンパイルし、本来真の表明が発火した。この `define` を外すと正常化する。
5. **確保ルーチンの経路** (特に厄介): mini-gmp モードの `mpfr_allocate_func` (`mpfr-gmp.c`) は `mp_get_memory_functions()` 経由で mini-gmp の確保フックを取得するが、これは**ホストの関数ポインタ**である。デバイスから呼ぶとゴミ (サイズをそのまま返す `0x28` 等) を返した。`__CUDA_ARCH__` 下で `mpfr_allocate/reallocate/free_func` がデバイスヒープ

(malloc/free) を直接呼ぶようにパッチした。

6. **定数キャッシュ** : `mpfr_const_pi/log2/euler/catalan` は計算関数ポインタ (ホストアドレス) を静的初期化で持ち、共有可変グローバルにメモ化する。デバイスでは違法命令・データ競合になる。これらを `MPFR_SAVE_EXPO` を伴うキャッシュレスのラップ `mpfr_const*_cuda` に振り替え、`*_internal` を直接呼ぶ (`SAVE_EXPO` を省くと値が誤る)。

3.3 最大の難関：デバイスでの `mpfr_div` 誤コンパイル

最も根深い問題は、`mpfr_div` の汎用経路 (精度 ≥ 3 リム) がデバイスで誤コンパイルされ `inf/ゴミを返す` ことであった (同一コードがホストでは正しい)。これは波及し、`const_pi/const_log2` が内部で除算するため誤値・クラッシュを生み、`log`, `sin`, `cos`, `atan`, `large exp` を一括して阻んだ (以前観測した `const_pi` 由来の `mpn_sqrtrem p[n-1]==0` 表明は、壊れた商が AGM の `mpfr_sqrt` に非正規値を供給した下流の症状であった)。

徹底的な二分切り分けで全ての低位プリミティブが単体ではデバイスで正しいことを確認した (`mpn_divrem_1`, `mpn_divrem`, `mini-gmp` の `mpz` 除算, `count_leading_zeros`, `udiv_qrnnd`, `MPFR_TMP`)。-O0/-G でも直らず、誤コンパイルされる構文の文単位の特定制も (計装書き込みが呼び出し前後で並べ替え・脱落して) 困難であった。そこで `nvcc` のコード生成問題を追うのをやめ、`mpz` プリミティブのみで構成した自己完結・正しく丸められる `mpfr_div` (`mpfr_div_cuda`) を代替実装した。被除数・除数の仮数を `mpz` に組み立て、ガードビット付きで 1 回 `mpz.tdiv_qr` を行い、`round/sticky` と剰余から `RNDN/Z/U/D/A` を丸め、指数を設定して `mpfr_check_range` する。公開 `mpfr_div` は `#ifdef __CUDA_ARCH__` でデバイスのみこれと呼ぶ。結果は `host mpfr_div` とビット一致 (7/2, 1/3, 7/1e9+7, 355/113@500b, 2/7@1024b で検証) し、これにより全超越関数がデバイスで動作した (`make test-mpfr-trans`: 128/128 スレッド一致)。

3.4 スタック・アリーナ・並列度

1. **スタックオーバーフロー** : `mpc_mul` $\rightarrow$ `mpfr_fmms` $\rightarrow$ `mpfr_sub` $\rightarrow$ `mpfr_sub1` のような深い呼び出し連鎖は、MPFR の大きなスタックフレームと相まって既定の約 1 KB の CUDA スレッドスタックを溢れさせる。`cudaDeviceSetLimit(cudaLimitStackSize, 128 KB)` で引き上げる。
2. **確保チャーン** : 要素ごとに多数のデバイス `malloc/free` が走るとその確保処理が支配的になる (アリーナ導入前は実 AXPY が 0.9 倍)。スレッド毎のバンプアリーナ (固定スラブを bump 確保, `free` は no-op, 作業項目ごとに reset) でこのチャーンを除去した。
3. **並列度不足** : 深い連鎖の中間リムは (グローバルメモリの) アリーナスラブに置かれるため、メモリレイテンシを隠すには多数の常駐ワープが要る。小さな起動では飢餓状態になり (MPC は $N=1024$ で 0.7 倍)。固定プールの $\sim 8\text{--}16\text{K}$ 常駐スレッドがベクトルをグリッドストライドする構成でレイテンシを隠し、約 40 倍を得た。

3.5 共存：cu_ 名前空間と umbrella ヘッダ

CPU 版の本物の libgmp/libmpfr/libmpc と同一バイナリに同居させるため、全ての公開シンボルを cu_ 名前空間へ機械的に改名した (mpfr_mul→cu_mpfr_mul など, tools/cu_prefix.py)。さらに型名・enum 定数・公開マクロ・include ガードもシステムヘッダと衝突するため、それらも cu_/CU_ で名前空間化した単一インクルードの **umbrella ヘッダ** (mpc_cuda.cuh) を tools/make_umbrella.py で生成した。これにより #include "mpc_cuda.cuh" 一行で、システムの <gmp.h>/<mpfr.h>/<mpc.h> と **同一翻訳単位** に置いても衝突しない。

3.6 その他のバグと教訓

- **mul.c のインライン PTX バグ**：src/mpfr_cuda/mul.c に手書き PTX の add_ssaaaa(..., 0, t) があり、整数リテラル 0 (4 バイト) を 64 ビット制約 "1" に渡してコンパイルエラーになっていた。(mp_limb_t)0 へキャストして解消。これが mul.o (cu_mpfr_mul) の欠落を招き、cu_mpfr/cu_mpc 系の再リンクを一時的に阻んでいた。
- **cudaLimitStackSize の戻り値**：GB10 では cudaLimitStackSize に設定可能な上限 (数百 KB) があり、512 KB の要求は cudaErrorInvalidValue で **拒否** される。戻り値を検査しないとスタックは既定の 1 KB のままとなり、深い cu_mpc 連鎖がスタックオーバーフロー (“illegal memory access” として観測) する。128 KB は受理され十分である。cudaDeviceSetLimit は必ず戻り値を検査せよ。

4 コンパイル時固定精度の高速パス

ランタイム精度の MPFR/MPC 移植とは別に、**精度をコンパイル時テンプレート引数で固定**することで仮数をレジスタに常駐させる高速パスを新設した。

4.1 設計

cu_fp::cu_freal<PB> (実数) と cu_fp::cu_fcomplex<PB> (複素) はヘッダオンリーのテンプレートである。PB は仮数ビット幅で **32 の倍数** (32, 64, 96, ..., 256, 512, 1024, ...) を取る。仮数は $\lceil PB/64 \rceil$ 個の 64 ビットリム配列で保持し、精度がコンパイル時定数のため schoolbook 乗算が完全アンロールされ、ptxas がローカル配列をレジスタに昇格する (global メモリのアリーナ往復が無い)。丸めは単一の finalize() ヘルパに集約し、sub-limb 境界 ($SB \in \{0, 32\}$) を含む任意精度を統一処理する。演算は + - * 演算子と from_double/(double) 変換を備える。実数の mul/add/sub は MPFR の RNDN と **ビット一致** し (PB=32–2048 で検証)、複素乗算は各成分を正しく丸めた実部 $\circ(a_r b_r - a_i b_i)$, 虚部 $\circ(a_r b_i + a_i b_r)$ を **完全な 2N リム積から 1 回だけ丸**

めて求め、MPC の MPC_RNDNN とビット一致する。

4.2 初等関数

`cu_fmath.cuh/cu_fcmath.cuh` は固定精度の初等関数を提供する：実数 `cu_{sqrt,cbrt,exp,expm1,log,log1p}`、複素 `cu_{cexp,clog,csqrt,csin,ccos,ctan,csinh,ccosh}`。方針は**ガードビット付き高精度近似**であり、作業精度 PB+128 ビットで Newton 反復 (`div/sqrt/cbrt`) と引数簡約級数 (`exp/log/三角/双曲`) を計算し PB に丸める。**正しい丸めは追求しない** (Table Maker’s Dilemma の困難ケースは非有界な Ziv ループを要し、固定精度・レジスタ常駐と相容れない)。MPFR/MPC との一致は ≤ 1 ULP、検証範囲 (PB=64-1024) では経験的に 0 ULP であった。特殊関数 (`gamma, zeta, erf, Bessel, Ai`) は今後の課題として除外した。

5 性能評価

主対象は NVIDIA GB10 (compute capability 12.1, CUDA 13, `-fmad=false`) であり、比較対象として NVIDIA H100 NVL (compute capability 9.0, `sm_90`) の結果を各項目で**並列に併記**する。特記なき限り精度は 1024 ビット、全結果はホストとビット一致 (初等関数は ≤ 1 ULP) である。2 台のホスト CPU が異なる (GB10 の Grace ARM と H100 機の x86) ため、GPU 同士の公平な指標は **GPU 時間 [ms]** であり、GPU 対 CPU 速度比は各々自ホストを基準とする参考値として併記する。各 GPU を直接対比するクロスプラットフォーム比較 (AXPY・線形計算・初等関数・精度スweepを通した総括) は、**本節の最後** (第 5.7 節) にまとめる。

5.1 AXPY (実数・複素)

ランタイム精度 MPFR/MPC 移植の AXPY $y = ax + y$ (N=16384, 1024 ビット) を GPU と CPU (本ライブラリのホスト経路) で比較した。同じデモを H100 でも実行し、GB10 と並べて表 2 に示す (GPU 時間で H100 が約 4 倍速い)。

表 2 ランタイム精度 AXPY (N=16384, 1024 ビット, 全ビット一致)。速度比は各々自ホスト CPU 基準 (GB10 : Grace ARM, H100 : x86)。

デモ	GB10 (sm_121)			H100 (sm_90)	
	GPU	CPU	速度比	GPU	速度比
MPFR 実数	1.17 ms	48.2 ms	41.3×	0.295 ms	143×
MPC 複素	4.81 ms	191.7 ms	39.9×	1.037 ms	160×

高速化にはバンプアリーナ (確保チャーン除去) と十分な常駐スレッド数 (メモリレイテンシ隠蔽) の双方が不可欠であった (第 3 節)。

5.2 基本線形計算（行列ベクトル積・行列積）

行列ベクトル積 $y=A \cdot x$ と行列積 $C=A \cdot B$ を実・複素で実装した（表 3）。内積のアクキュムレータ・スクラッチを**スタック常駐**（MPFR_DECL_INIT / 複素は mpfr_custom_init_set ベースの MPC_DECL_INIT）とし、アリーナは 1 回の積和ぶんのみ保持して内側反復ごとに reset する。出力要素をグリッドストライドするため $O(N^2)$ の行列積がデバイスを飽和させ、行列積では 100 倍超を得た。全結果ビット一致。同じカーネルを H100 でも実行し、GB10 と並べて表 3 に示す。H100 は飽和する行列積で速い一方、小 N の行列ベクトル積では占有率不足で GB10 に劣る（詳細は第 5.7 節）。

表 3 多倍長線形計算（1024 ビット，全ビット一致）。速度比は各々自ホスト CPU 基準。

カーネル	N	GB10 (sm_121)			H100 (sm_90)	
		GPU	CPU	速度比	GPU	速度比
MPFR 行列ベクトル積	256	11.3 ms	177 ms	15.6×	15.80 ms	9.2×
MPFR 行列積	96	23.0 ms	2.41 s	105 ×	12.00 ms	175 ×
MPC 行列ベクトル積	128	24.4 ms	179 ms	7.3×	29.68 ms	5.2×
MPC 行列積	64	27.9 ms	2.88 s	103 ×	15.63 ms	155 ×

5.3 初等関数

$N=4096$ 入力に対し関数群を評価した（GPU vs CPU，全関数ビット一致）。

- **MPFR**: sqrt cbrt exp expm1 log log1p sin cos tan atan sinh cosh $\Rightarrow$ GPU で **42–77**×
- **MPC**: sqr sqrt exp log sin cos tan sinh cosh asin acos atan $\Rightarrow$ GPU で **22–63**×

関数別の内訳を表 4・図 1 に並列に示す。**H100 は関数別の実測値**（GPU vs 自ホスト CPU）を掲げ、**GB10 は同一関数群の範囲**（上記の MPFR: 42–77×，MPC: 22–63×）を**灰色帯として併記**する（GB10 の関数別計測は本機では再取得できないため範囲で示す）。両者は同形により広い分布を示し、軽量カーネル（sqrt・cbrt・sqr）は GPU 側が短時間で済む一方ホスト MPFR/MPC はセットアップ支配のため 170–217× に達し、多段 AGM を要する重いカーネル（log，複素 asin/acos）が下端となる。なお GB10 と H100 の速度比はそれぞれ自機の CPU を基準とするため比として直接の比較はできない。

表 4 初等・超越関数の GPU 速度比 (N=4096, 1024 ビット, 全ビット一致)。H100 は関数別の実測, GB10 は同一関数群の範囲 (MPFR: 42–77×, MPC: 22–63×) を併記。

MPFR (実数)				MPC (複素)			
関数	GPU	CPU	速度比	関数	GPU	CPU	速度比
sqrt	0.86	185.9	217×	sqr	0.24	42.6	180×
cbrrt	3.51	597.7	170×	sqrt	2.93	362.6	124×
exp	10.60	1271.3	120×	exp	23.71	3255.8	137×
expm1	9.69	1305.7	135×	log	77.46	6378.9	82×
log	39.71	4189.5	105×	sin	24.79	1398.0	56×
log1p	25.99	3510.6	135×	cos	24.75	1396.7	56×
sin	3.14	362.1	115×	tan	36.80	1583.7	43×
cos	2.05	219.4	107×	sinh	24.66	1422.3	58×
tan	3.37	380.3	113×	cosh	24.61	1420.5	58×
atan	21.75	1834.1	84×	asin	87.69	6886.2	79×
sinh	20.75	957.1	46×	acos	94.98	7511.9	79×
cosh	14.32	936.4	65×	atan	121.12	11179.5	92×

時間は ms。参考: GB10 (同一関数群) の速度比範囲は MPFR: 42–77×, MPC: 22–63× (関数別の内訳は灰色帯として図 1 に併記)。

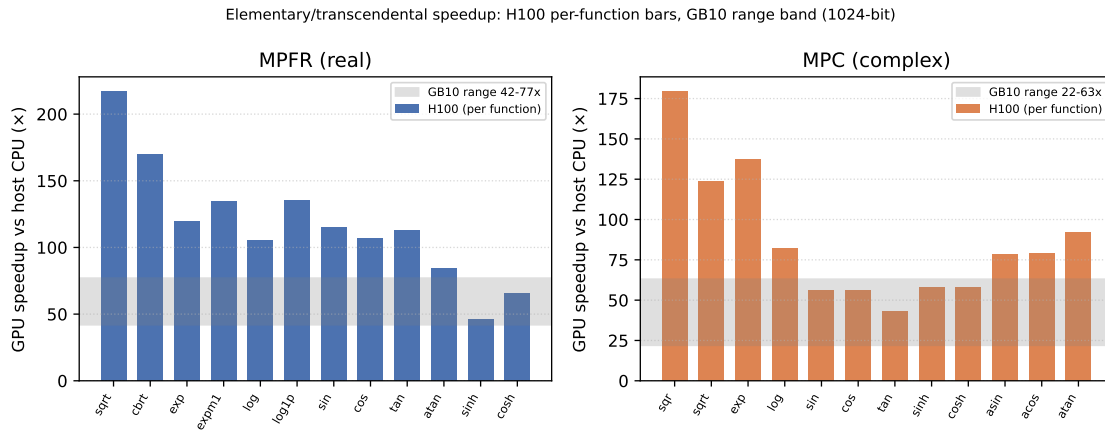


図 1 初等・超越関数の GPU 速度比 (対 自ホスト CPU)。棒 = H100 の関数別実測, 灰色帯 = GB10 の範囲 (MPFR: 42–77×, MPC: 22–63×)。左: MPFR (実数), 右: MPC (複素)。1024 ビット, 全ビット一致。

5.4 固定精度高速パスと精度スweep

固定精度パスはランタイム精度パスより大幅に速い。1024 ビット AXPY (N=2²⁰) で `cu_freal` は既存の `cu_mpfrr-GPU` 経路の **142 倍**, system CPU MPFR の **352 倍**であった (いずれもビット一致)。

精度依存を明らかにするため, AXPY を掃引した (`make bench3`)。GB10 は 128–8192 ビット (表 5), H100 は固定・ランタイム・CPU の各経路が**完全に交差するまで 128–65536 ビット**へ

延長した（表 6）。GB10 と H100 を重ねた時間・速度比を図 2（実数）・図 3（複素）・図 4（速度比）に示す（実線＝H100, 破線＝GB10）。

GB10 では `cu_freal` の時間は 1024 ビットまでほぼ一定（完全レジスタ常駐）、**2048 ビットで約 7 倍に跳ね上がる**（仮数 $N = 32$ リムでレジスタに収まらずローカルメモリへスピルする境界）、8192 ビットで CPU 1 コアと同等になる。複素も同形でより大きなリードを示す（1024 ビットで `cu_mpc` 比 約 48×, CPU 比 約 107×）。

表 5 固定精度パスの精度スweep (AXPY, $N=4096$, GB10, 全ビット一致)

ビット	<code>cu_freal</code>	<code>cu_mpfr</code>	CPU	<code>freal/cu_mpfr</code>	<code>freal/CPU</code>
128	0.005	0.050	0.190	10.2×	38.5×
256	0.005	0.088	0.222	19.6×	49.1×
512	0.006	0.174	0.320	29.5×	54.1×
1024	0.008	0.388	0.677	47.4×	82.6×
2048	0.056	0.506	0.322	9.0×	5.7×
4096	0.184	0.996	0.473	5.4×	2.6×
8192	0.823	1.989	0.814	2.4×	1.0×

時間は ms (8 回平均)。

H100 では大きなマシンがレイテンシをよく隠すため、第 5.7 節の占有率の議論と整合する 2 つの効果が現れる（表 6）。第一に、**GPU ランタイム経路** (`cu_mpfr/cu_mpc`) は**全精度で H100 が GB10 より約 2–5× 速い**（例：1024 ビット実数で 0.120ms 対 GB10 の 0.388ms）。第二に、H100 の**固定精度スピル境界（2048 ビット）ははるかに緩やか**で、`cu_freal` は 1024→2048 ビットで $\sim 2.75\times$ (0.008→0.022ms) しか上がらない（GB10 は $\sim 7\times$, 0.008→0.056ms）。これは H100 の大きなレジスタファイルとローカルメモリ帯域がレジスタ → ローカルのスピルを緩和するためである。

■**全経路の交差 (H100)** ビット長をさらに伸ばすと固定精度の優位は完全に消える。実数では `cu_freal` が **8192 ビットで `cu_mpfr` とパリティ** (0.430 対 0.436ms), **16384 ビットで CPU とも交差** (2.477 対 2.847ms) し、以降は急激に最悪化する (32768 ビットで 64.6ms, 65536 ビットで 482ms)。これはスピルした *schoolbook* $O(n^2)$ がローカルメモリを激しく往復するためである。複素も同形で、`cu_fcomplex` は 16384 ビットで `cu_mpc` と、32768 ビット手前で CPU と交差する。一方 **GPU ランタイム経路は 65536 ビットでも CPU より速く** (実数 3.6 対 6.4ms, 複素 31.4 対 60.9ms), 高精度では固定精度ではなくランタイム（あるいは劣二次乗算を持つ CPU）が適することを示す。全スweep点でホストとビット一致 (`rel=0`)。

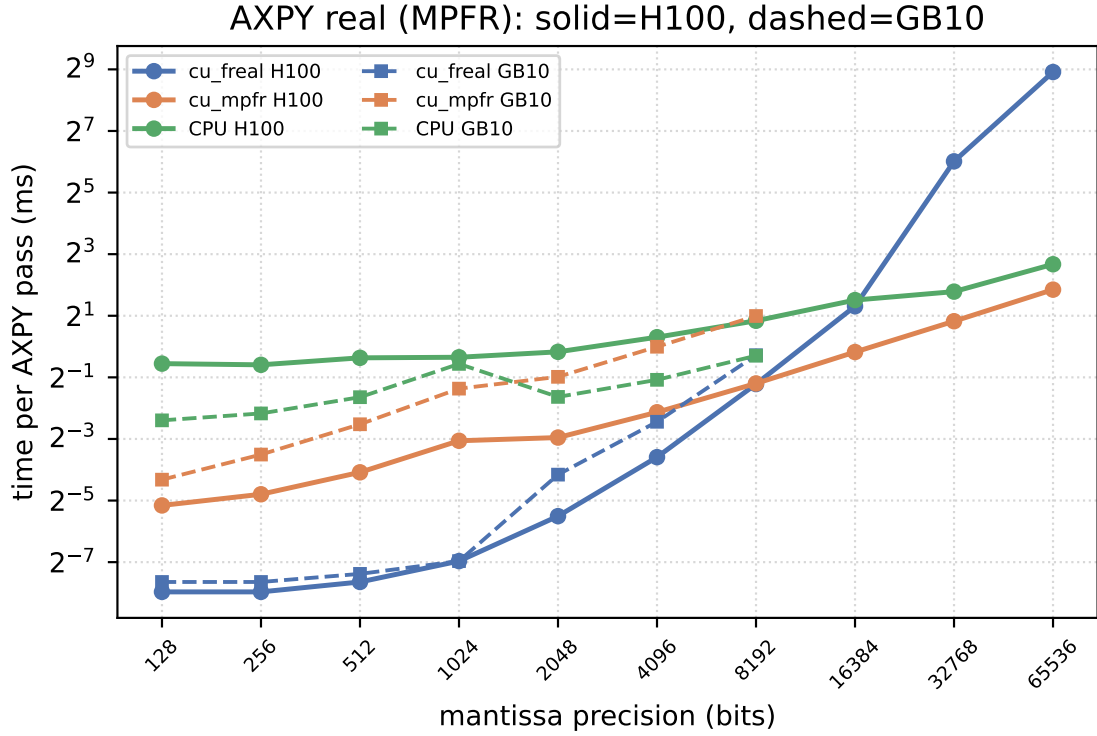


図2 実数 AXPY の時間 vs 精度 (両対数)。実線 = H100 (65536 ビットまで), 破線 = GB10 (8192 ビットまで)。青 = `cu_freal`, 橙 = `cu_mpfr`, 緑 = CPU。固定精度は 1024 ビットまで一定で, 2048 ビットでスピルにより跳ね上がり, H100 では 8192 ビットで `cu_mpfr` と, 16384 ビットで CPU と交差し, 以降急激に最悪化する。

5.5 CPU 上での固定精度パス

`cu_freal/cu_fcomplex` はヘッダオンリーで, ホストでは `__int128` を用いて `g++` のみでコンパイル・実行できる。CPU 単体 (1 スレッド) でシステム MPFR/MPC と比較すると, **低精度でのみ速い** (表 7)。`cu_freal` の乗算は schoolbook $O(n^2)$ だが GMP/MPFR は劣二次 (Karatsuba/Toom/FFT) であり, GPU で勝てたのはレジスタ常駐と大量並列で $O(n^2)$ を隠せたからである。CPU 単スレッドにはその利点が無いため, 約 512 ビットを超えると算法差がそのまま現れる。

5.6 考察

GPU では固定精度レジスタ常駐パスが **~1024 ビットまで圧勝**し (CPU 比最大 ~83×, `cu_mpfr`-GPU 比 ~142×), 2048 ビットでスピル境界 (knee) を迎え, 8192 ビットで CPU と同等になる。一方 CPU 単体では同パスは 128–256 ビットでのみ MPFR/MPC を上回る。すなわち固定精度 schoolbook の優位は (i) **GPU の並列性**, (ii) **低~中精度**に限られ, 高精度・CPU

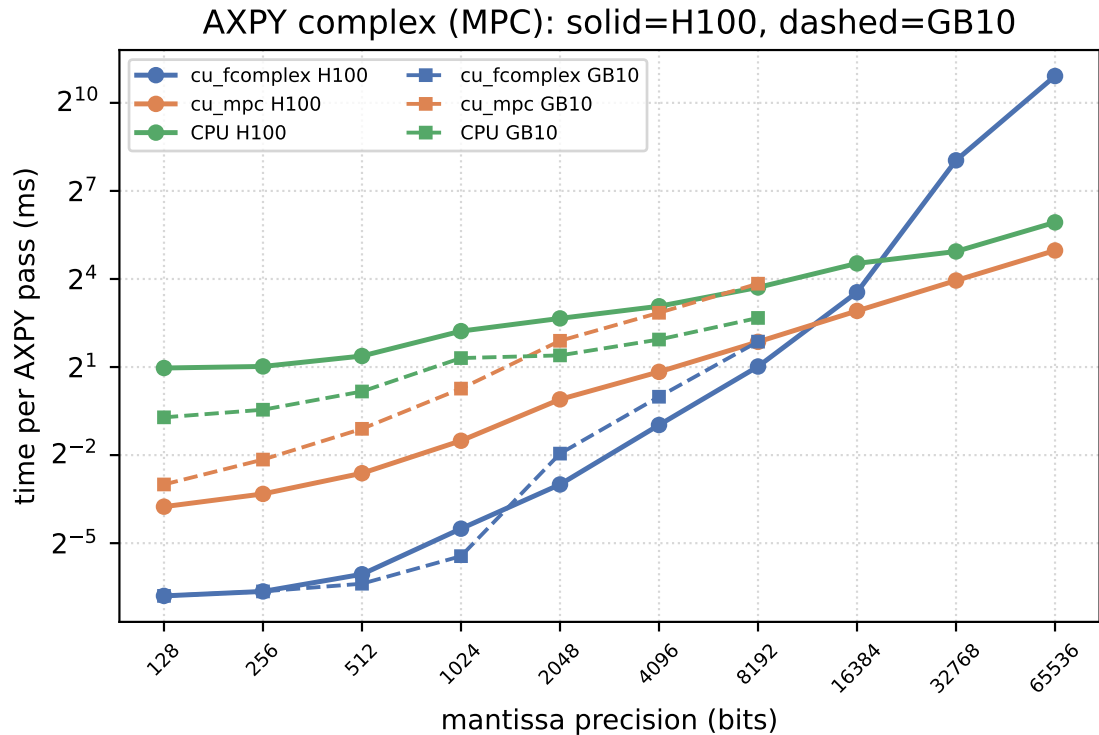


図3 複素 AXPY の時間 vs 精度 (両対数)。系列は図2と同様。実数と同形で固定精度のリードはより大きいが、H100 では `cu_fcomplex` が 16384 ビットで `cu_mpc` と、32768 ビット手前で CPU と交差する。

では劣二次乗算を備えた MPFR/MPC が適する。

5.7 クロスプラットフォーム比較：GB10 vs H100（総括）

最後に、本節で個別に併記してきた結果を2つのGPUの観点から総括する。ランタイム精度の6デモ（実・複素 AXPY、行列ベクトル積、行列積）は同一ソース・同一問題サイズ・1024ビットで両機で実行され、全てビット一致である。2台の宿主CPUが異なる（GB10のGrace ARMとH100機のx86）ため、**GPU 時間 [ms]** がハードウェア同士の公平な指標であり、GPU vs CPU 速度比は参考として併記する（表8・図6）。

この傾向は占有率 (occupancy) で一貫して説明できる。H100 は**高並列**カーネル—AXPY (16384要素のメモリバンド幅律速で $\sim 4\times$)と行列積 (デバイスを飽和させる $O(N^2)$ グリッドで $\sim 1.8\text{--}1.9\times$)—で、132基のSMとHBM3バンド幅を活かして圧倒する。一方、行列ベクトル積では**負ける** ($0.72\text{--}0.82\times$)：出力1行あたり1スレッドで $N=128\text{--}256$ ゆえ常駐スレッドは高々 $\sim 128\text{--}256$ にとどまり、起動が**占有率不足**となって1本の長い多倍長内積のシングルスレッドレイテンシ律速となる。この領域では新しいGB10コア (sm_121, 高クロック) が速い。図7に2機のGPU vs CPU 速度比を並べて示す。

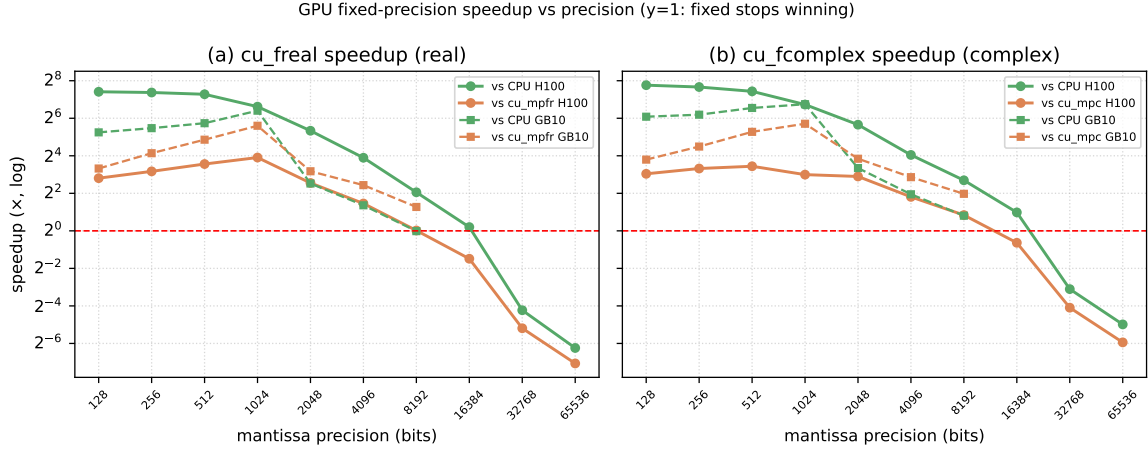


図 4 GPU 固定精度パスの速度比 vs 精度 (両対数, $y = 1$ がパリティ)。実線 = H100, 破線 = GB10。 (a) 実数 (対 CPU・対 cu_mpfpr), (b) 複素 (対 CPU・対 cu_mpc)。1024 ビットでピーク, 2048 ビットがスピル境界 (knee), 高精度で $y < 1$ となり固定精度が負ける。

表 6 H100 での精度スweep (AXPY, $N=4096$, 全ビット一致)。GB10 は本機で再取得できないため 8192 ビットまでの表 5 を参照。時間は ms (8 回平均)。

ビット	実数 (MPFR)			複素 (MPC)		
	cu_freal	cu_mpfpr	CPU	cu_fcplx	cu_mpc	CPU
128	0.004	0.028	0.681	0.009	0.074	1.955
256	0.004	0.036	0.663	0.010	0.100	2.029
512	0.005	0.059	0.776	0.015	0.163	2.595
1024	0.008	0.120	0.786	0.044	0.351	4.670
2048	0.022	0.129	0.888	0.125	0.932	6.309
4096	0.083	0.228	1.235	0.509	1.785	8.403
8192	0.430	0.436	1.787	2.025	3.626	13.105
16384	2.477	0.884	2.847	11.693	7.529	23.115
32768	64.635	1.769	3.443	263.499	15.429	30.636
65536	482.420	3.601	6.375	1929.433	31.360	60.912

太字相当の転換点: 実数は 8192 ビットで $\text{cu_freal} \approx \text{cu_mpfpr}$, 16384 ビットで $\text{cu_freal} > \text{CPU}$ 。複素は 16384 ビットで $\text{cu_fcplx} \approx \text{cu_mpc}$ 。

初等関数 (表 4・図 1) も同じ構図を示す: $N = 4096$ の関数評価はデバイスを十分に埋めるため両機で高速で, H100 は関数別に **43–217**× (対自ホスト x86), GB10 は **42–77**× (MPFR) / **22–63**× (MPC, 対 Grace ARM) に分布する。速度比は各々自ホスト基準ゆえ直接比較はできないが, 両機とも軽量カーネル ($\text{sqrt} \cdot \text{cbrr} \cdot \text{sqr}$) で上端, 多段 AGM の重いカーネル ($\log \cdot \text{複素} \text{ asin/acos}$) で下端という同形である。

精度スweep (第 5.4 節) でも同じ占有率効果が一貫する: H100 は $N = 4096$ でレイテンシをよく隠すためランタイム経路が全精度で約 2–5× 速く, 固定精度のスピル境界 (2048 ビット) も緩やかであった (GB10 ~7× に対し H100 ~2.75×)。

要するに H100 は**埋めるだけの並列性がある時** (高並列 AXPY・行列積・初等関数・ N の大き

表 7 CPU 単体での固定精度 vs システム MPFR/MPC (AXPY, N=4096, 1 スレッド, 速度比 > 1 で固定精度が速い)

ビット	freal/MPFR	fcplx/MPC
128	1.28×	1.79×
256	1.23×	1.48×
512	1.03×	0.99×
1024	0.71×	0.67×
2048	0.09×	0.17×
4096	0.03×	0.06×
8192	0.01×	0.03×

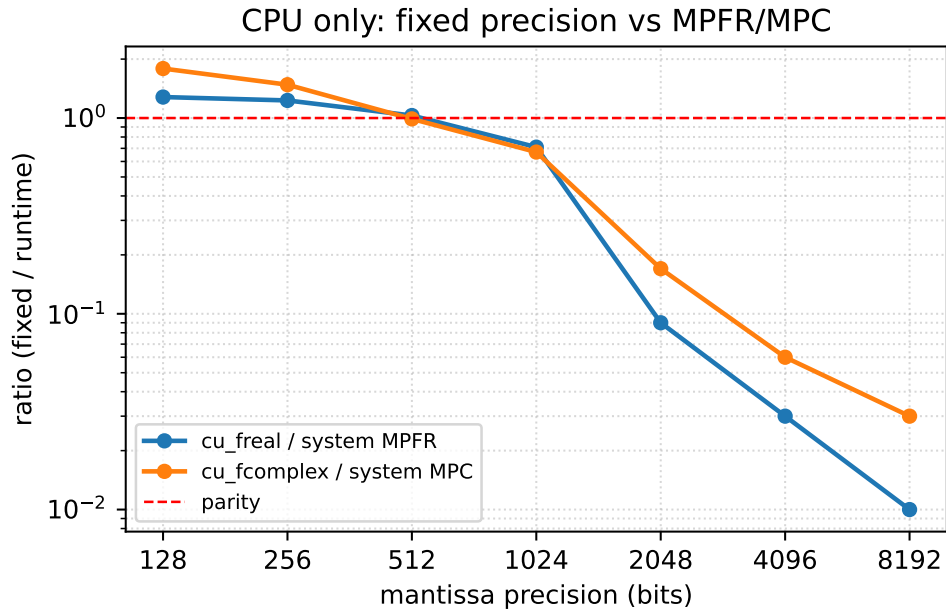


図 5 CPU 単体での固定精度 vs MPFR/MPC の比 (縦軸対数, $y = 1$ がパリティ)。固定精度が上回るのは 128–256 ビットのみで、高精度では劣二次乗算を持つ MPFR/MPC が勝つ。

いスイープ) に優れ, GB10 は**レイテンシ律速の小 N 領域** (行列ベクトル積) で競争力を持つ。

6 まとめと今後

GMP(mini-gmp)/MPFR/MPC を, 再実行可能な変換スクリプトにより**忠実に** CUDA カーネル内へ移植し, 実数 MPFR・複素 MPC の AXPY を CPU とビット一致のまま約 40 倍高速化した。移植の核心は, コンパイル可能化 (NORETURN/NO_ASM/realloc/_Complex 等), デバイス実行可能化 (RODATA/managed グローバル/確保経路/定数キャッシュ), そして mpfr_div

表 8 GB10 vs H100, ランタイム精度デモ (1024 ビット, 全ビット一致)。比は GB10/H100 の GPU 時間 (> 1 で H100 が高速)。

デモ	N	GB10 GPU	H100 GPU	H100/GB10
AXPY MPFR	16384	1.17 ms	0.295 ms	3.97×
AXPY MPC	16384	4.81 ms	1.037 ms	4.64×
行列ベクトル積 MPFR	256	11.3 ms	15.80 ms	0.72×
行列ベクトル積 MPC	128	24.4 ms	29.68 ms	0.82×
行列積 MPFR	96	23.0 ms	12.00 ms	1.92×
行列積 MPC	64	27.9 ms	15.63 ms	1.79×

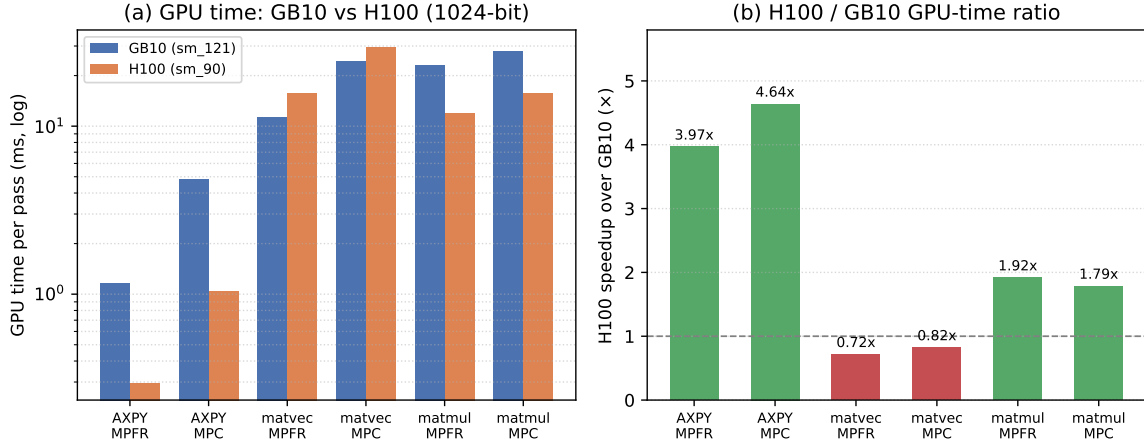


図 6 GB10 vs H100。(a) パスあたり GPU 時間 (対数軸), (b) H100 の GB10 比速度 ($y = 1$ がパリティ)。H100 は高並列カーネルで勝ち、小 N の行列ベクトル積で負ける。

のデバイス誤コンパイルに対する mpz ベース代替であった。さらにスタック上限・アリーナ・並列度の調整, cu_ 名前空間と umbrella による共存を整えた。

加えて, コンパイル時固定精度のレジスタ常駐型 `cu_freal<PB>/cu_fcomplex<PB>` と固定精度初等関数を新設した。これらは GPU の低～中精度で MPFR/MPC を桁違いに上回り, 精度スweepにより **2048 ビットがスピル境界, 8192 ビットで CPU 同等**というクロスオーバーを定量化した。さらに GB10 と H100 NVL のクロスプラットフォーム比較により, ランタイム精度カーネルが**占有率律速**であることを示した: 並列性が 132 基の SM を埋められる AXPY ($\sim 4\times$)・行列積 ($\sim 1.8\text{--}1.9\times$) では H100 が高速だが, 小 N の行列ベクトル積はレイテンシ律速のままで GB10 がわずかに優る。

今後の課題は, MPFR/MPC の特殊関数 (gamma, zeta, erf, Bessel 等) の固定精度実装, 高精度域のレジスタ圧・コンパイル時間の削減, 固定精度向けの劣二次乗算 (Karatsuba 等) の導入, そして per-thread の指数・フラグ状態による完全な競合排除である。

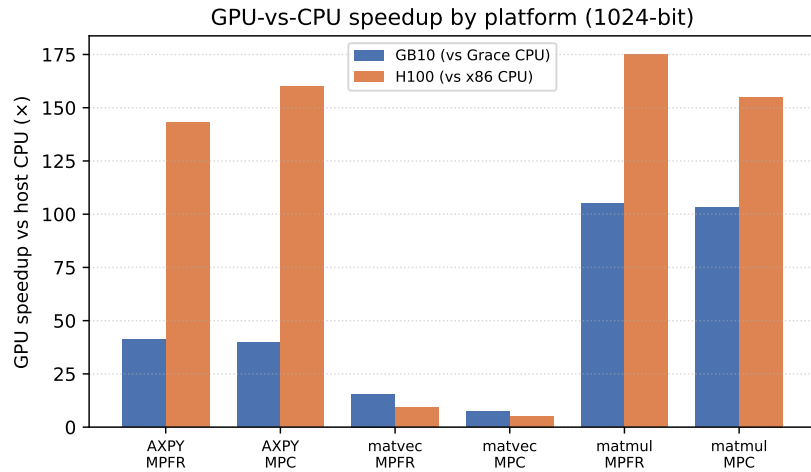


図7 プラットフォーム別の GPU vs CPU 速度比 (各 GPU は自ホスト CPU 基準), 1024 ビット。H100 の高い AXPY/行列積速度比は, 速い GPU と異なる (x86) ホスト基準の双方を反映する。

再現方法: `./configure && make ; make check` (minigmp/mpfr/mpc/trans のビット一致), `make demos/matrix/bench` (性能), `make sample-fixed/check-fixed/check-fmath/bench3` (固定精度)。CPU 参照ターゲット (`bench3・coexist・cputest・cpubench`) はシステムの GMP/MPFR/MPC を要する。非標準のプレフィックスにある場合は `./configure --with-mp-prefix=DIR` (例: `~/local`) を指定すると, その `include/lib` (rpath 付き) がビルドに加わる。対象 GPU は `--with-cuda-arch=ARCH` (既定 `sm_90`; GB10 の測定は `sm_121`) で選ぶ。詳細はユーザマニュアル `doc/manual.ja.md` (日本語) / `doc/manual.md` (英語) を参照。